MAVRA: A Shared Memory for Multi-agent Workloads

Author Names (to be filled in)
Affiliation

***Abstract*—Data agents repeatedly explore the same database across tasks and sessions, spending LLM calls on knowledge that previous agents have already discovered. We present MAVRA, a shared memory layer between data agents and the database that reduces exploration, token consumption, and end-to-end time. The layer extracts executable metric and feature definitions from successful sessions, shares them across agents, and compiles them for new questions and prediction instances. Learning, validation, and optimization costs are amortized across consumers. On a TPC-DS workload with 16 agents, MAVRA reduces token consumption by 72% and achieves a 4.6× end-to-end speedup relative to no memory, while using 65% fewer tokens than private memory. On six RelBench retraining tasks, token use from the second cutoff onward falls by 69% on rel-stack and 50% on rel-trial relative to no memory, with comparable predictive quality. The layer also improves and maintains correctness through publication checks, snapshot-bound condition validation, and shared maintenance. TPC-DS accuracy increases from 74% to 96% initially and reaches 99% after a data restatement, compared with 55% without memory.**

***Index Terms*—data agents, shared memory, metric definitions, feature engineering, data validation**

I. Introduction

Data agents repeatedly explore the same database to answer different questions and build predictive features. An agent asking for March revenue, another asking for September revenue, and a third asking for quarterly revenue all need the same measure, grain, filters, and joins. When each session discovers these independently, the workload pays repeatedly for schema inspection, value probes, join tests, and the LLM calls that plan and interpret them. In our traces of 100 memoryless metric sessions, 93% of lookups repeat a fact obtained by an earlier session, and 71% of session time precedes the answer query. Public benchmark tasks exhibit the same overlap in calculations and domain knowledge (Section II-A).

We present MAVRA (**M**etric-**A**ware **V**alidation and **R**euse for **A**gents), a shared memory layer between data agents and the database that reduces repeated exploration, token consumption, and end-to-end time. Figure 1 contrasts independent sessions with this shared layer. Successful sessions contribute database knowledge; later agents retrieve it and obtain SQL for their own periods or prediction instances. Knowledge accumulates around the database and serves multiple agents, tasks, and sessions. A new consumer can therefore use discoveries made by previous consumers, amortizing learning over the workload.

The layer stores executable metric and feature definitions, together with table descriptions and verified join paths. A definition captures a measure, its grain, joins, and required filters;

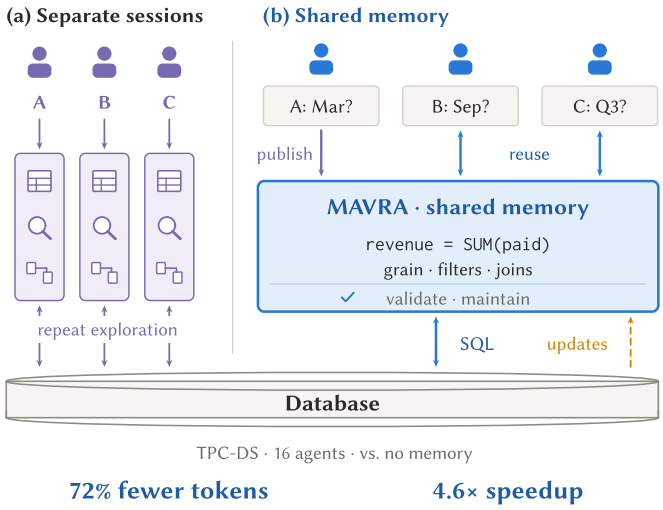


Fig. 1. Shared memory reduces repeated exploration. (a) Each agent inspects schemas, probes values, and tests joins. (b) MAVRA sits between agents and the database: successful sessions contribute definitions; later agents reuse them. Validation and maintenance are shared within the layer. Reported savings are from the 16-agent TPC-DS study (Section V-C).

feature definitions additionally encode the entity and temporal window. Compilation binds these objects to a new question or seed table. On a definition hit, compilation bypasses repeated schema, value, and join exploration, shortening the sequence of LLM and tool calls and the contexts processed by those calls. Our cost model makes the effect explicit: as more questions reuse a learned definition, the initial discovery and publication cost is spread over more uses, and per-question cost approaches the cost of reuse (Section II-B).

The shared layer also improves and maintains answer correctness. Publication checks verify extracted definitions before other agents use them. Conditions derived from grain and joins are checked on the consuming query's snapshot, and their results are shared by dependency version. Data changes trigger revalidation, checked metric repair, or revocation; feature compilation enforces per-instance temporal bounds. These mechanisms let consumers share validated knowledge and its upkeep along with the savings from reuse (Section IV).

We evaluate token and time savings on RelBench [1], TPC-DS, and TPC-H. On the 16-agent TPC-DS workload, MAVRA uses 72% fewer tokens and achieves a 4.6× end-to-end speedup over no memory; it uses 65% fewer tokens

than private memory with the same learning mechanisms. For later questions, LLM calls fall from 6.9 to 3.2 and tool calls from 10.6 to 4.4. On RelBench, token use from the second retraining cutoff onward falls by 69% on rel-stack and 50% on rel-trial relative to no memory, and by 52% on both databases relative to the most token-efficient agent-memory baseline, with comparable predictive quality across six tasks. TPC-DS accuracy rises from 74% to 96% initially and reaches 99% after a restatement, compared with 55% without memory.

The contributions are:

- A shared memory layer between agents and the database that turns session discoveries into executable metric and feature definitions, reducing repeated exploration across agents and tasks (Sections III and IV).
- A workload analysis, amortization model, and end-to-end evaluation that connect shared knowledge reuse to reductions in LLM calls, tokens, and time (Sections II and V).
- Publication validation, snapshot-bound shared checks, and maintenance mechanisms that improve answer correctness and govern reuse after data changes (Section IV).

II. Background and Motivation

A. Repeated knowledge across agent queries

Tasks over the same database require overlapping domain knowledge. We quantify this overlap using the released tasks of public text-to-SQL benchmarks. LiveSQLBench [2], on which BIRD-Interact builds [3], documents each of its PostgreSQL databases in a knowledge base of calculations, domain rules, and value meanings (1,369 entries for the 22 databases of the full release), and BIRD [4] gives each question evidence that defines its terms. Since LiveSQLBench withholds which entries a task needs, we count a task as needing an entry when its question names the entry or an entry built on it; a BIRD question needs the terms that its evidence defines. By excluding implicit references and paraphrases, these lexical counts provide a conservative estimate of knowledge repetition. Table I shows that most tasks share knowledge with another task on their database, 78% on LiveSQLBench and 51% on BIRD. Half of LiveSQLBench's uses of knowledge (50%) repeat an earlier use, and calculations, the kind of knowledge that MAVRA learns, repeat as often (53%). Repetition grows with the number of tasks: the 10th task on a LiveSQLBench database finds 48% of the knowledge it needs already needed by an earlier task, and the 20th 65%; on BIRD the share still grows to 33% by the 100th question. These benchmarks contain 27 to 139 tasks per database on average. Every task is answered by a new agent session, which explores the knowledge again.

In applications, parameterized query templates and database knowledge reside in code, so millions of queries reduce to thousands of templates [5]; a new data-agent session must acquire that knowledge through schema lookups, data probes, and checks. To see what this costs, Table II compares two workloads. The application side is Redset [6]: 441 million queries of 400 Amazon Redshift clusters over three months, analyzed per cluster with the dataset's template fingerprint [7]. To capture session-level execution and LLM overhead not logged in the static benchmarks, the agent side traces 100 memoryless DeepSeek-V4.1-Flash-1017 sessions over 25 analytical questions spanning five metrics (Section V: the learning questions and the first three scored forms). Each session uses table-listing, table-description, and SQL tools. A lookup's *fact* is its purpose together with the tables and columns it reads.

Applications repeat queries: in Redset, 93.4% of the statements repeat an earlier template, 6.6 templates per 100 statements, and prepared statements and the result cache, which answers 34% of the reads, serve them well. Agent queries have greater template diversity in our traces: 100 agent statements contain 76 templates, because each session writes its SQL

TABLE I

Knowledge that tasks on one database share in public benchmarks. A LiveSQLBench task needs the knowledge-base entries that it names and the entries they build on; a BIRD question needs the terms that its evidence defines. Shares of tasks are of the tasks that need knowledge; the last rows average over random arrival orders of the tasks on each database.

	LiveSQLBench		BIRD
	Full	Lite	dev
Databases	22	18	11
Tasks	600	270	1,534
needing knowledge	82%	97%	83%
sharing it with another task	78%	78%	51%
Uses of knowledge repeating an earlier use	50%	40%	28%
among calculations	53%	47%	26%
Knowledge of the n-th task already needed			
$n = 10$	48%	47%	15%
$n = 20$	65%	–	24%
$n = 100$	–	–	33%

TABLE II

Application and agent workloads. Application: Redset [6], 441 M queries from 400 Redshift clusters (read queries). Agents: 100 new sessions issuing 521 database calls, 176 of them SQL; a *lookup* is a schema lookup, data probe, or validation call; an LLM call issues no answer SQL when none of its tool calls runs a query that the final answer uses.

	Redset	Agents
LLM calls per session	–	4.6
issuing no answer SQL	–	57%
Session time before the answer SQL	–	71%
Lookups repeating an earlier fact	–	93%
share of lookup DB time	–	77%
SQL lookups repeating a fact	–	38%
with the same SQL text	–	0%
SQL repeating an earlier template	93.4%	24%
Templates per 100 statements	6.6	76
Answered from the result cache	34%	–
SQL that fails	0.35%	1.7%
Wrong answers from SQL without error	–	32 / 32
Calls that validate	none	4.8%
Sessions checking key uniqueness	–	16%